JDBC dan Koneksi Java–MySQL



D

I

S

U

S

U

N

Oleh :

Nama : Putri Sibarani

Nim : 25110183

Pertemuan 6

Matkul : Pemrograman Aplikasi Client Server

Program studi : D3 Teknik Komputer

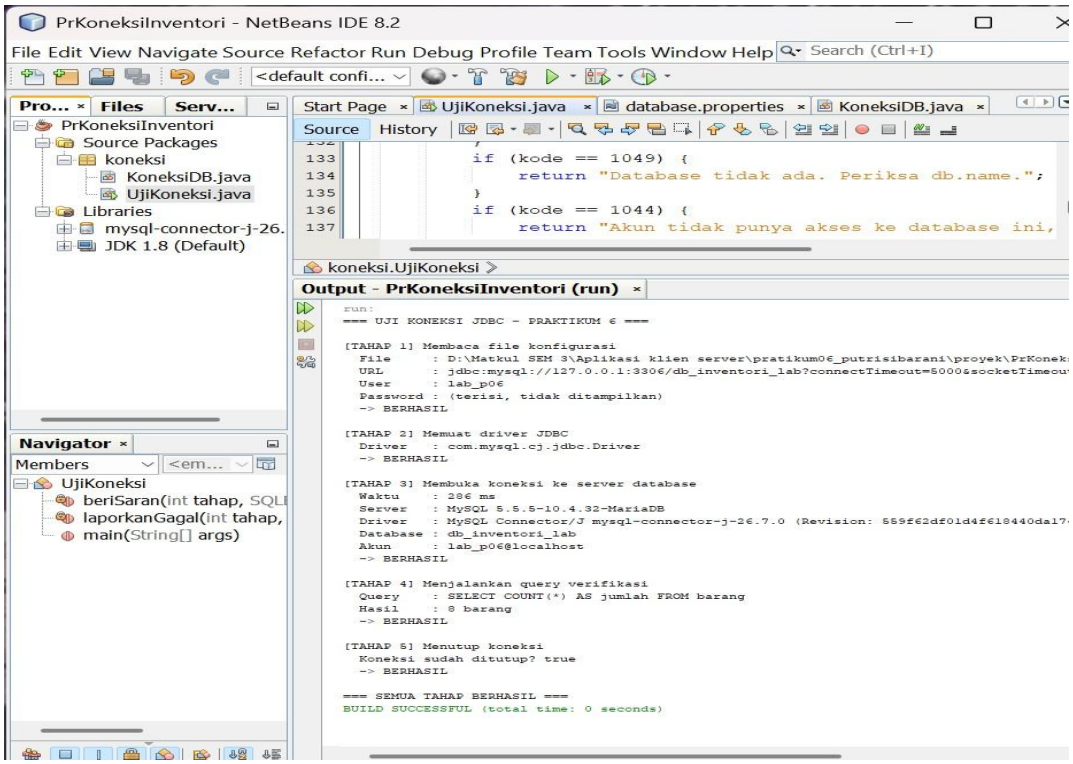
Dosen Pengampu : Juni Ismail, S.Kom., M.Kom.

Politeknik Bisnis Indonesia

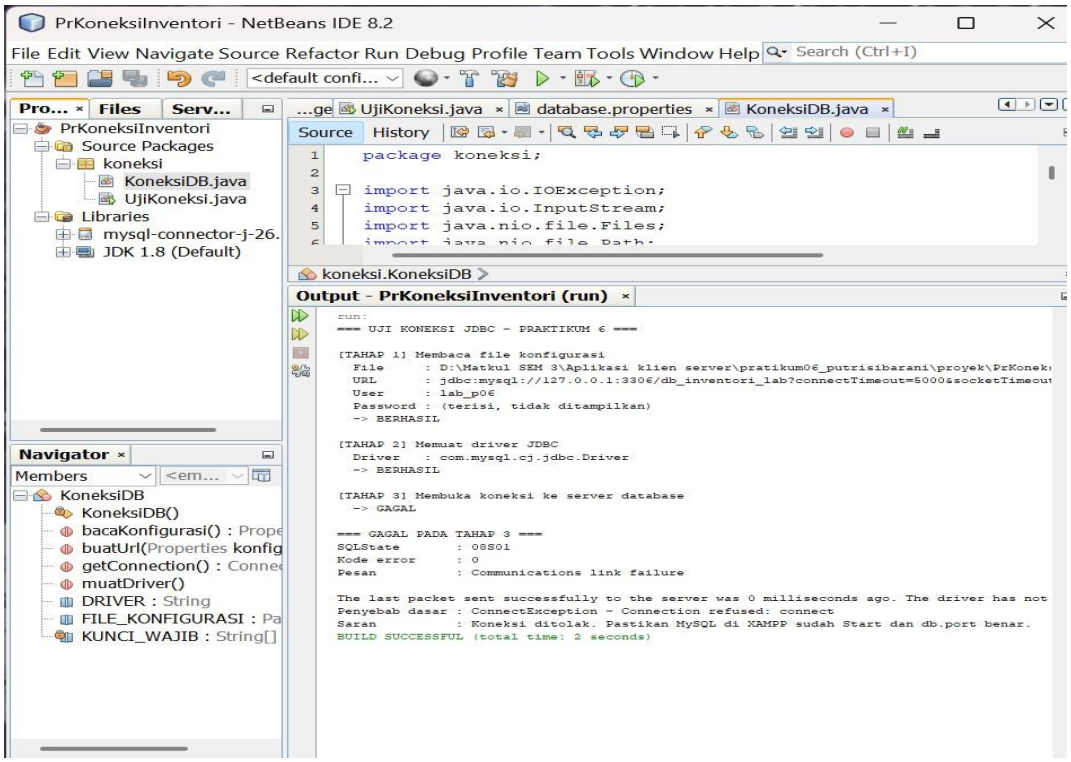
2026/2027

JDBC DAN KONEKSI JAVA

1. Hasil output pengujian di 5 tahap yang berhasil

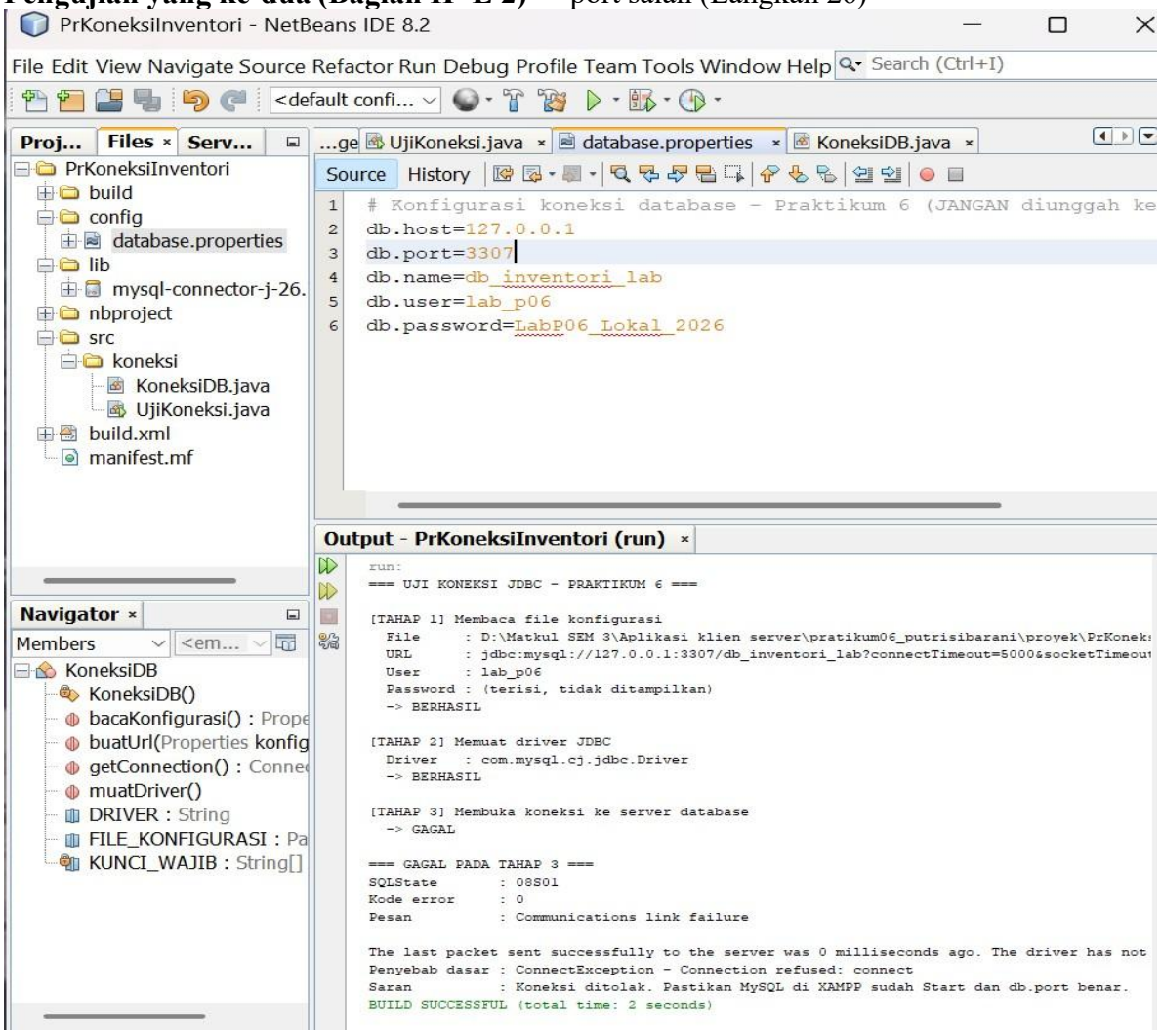


2. **Pengujian yang pertama (Bagian H -E 1) ->** Layanan MySQL Mati (Langkah 25)



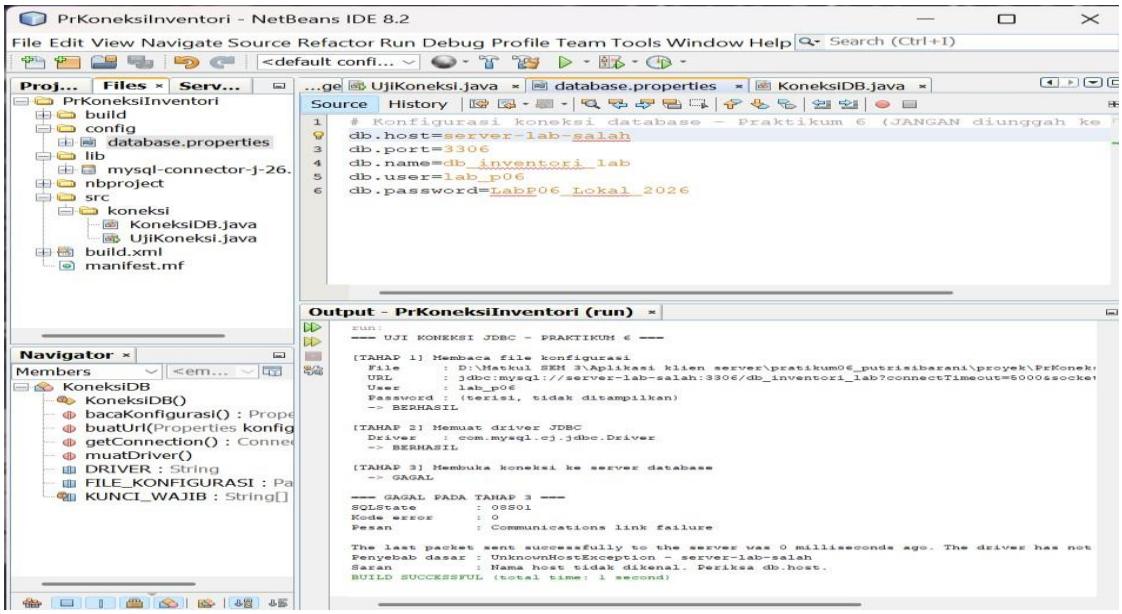
Ubah	Di XAMPP Control Panel, klik **Stop** pada baris MySQL. Tunggu sampai latar nama MySQL tidak hijau lagi.
Hasil yang diharapkan	Gagal di **Tahap 3**; SQLState 08S01; penyebab dasar ConnectException - Connection refused: connect
Kembalikan	Klik **Start** pada baris MySQL, tunggu hijau, lalu tekan **F6** sampai berhasil

3. **Pengujian yang ke-dua (Bagian H -E 2) ->** port salah (Langkah 26)



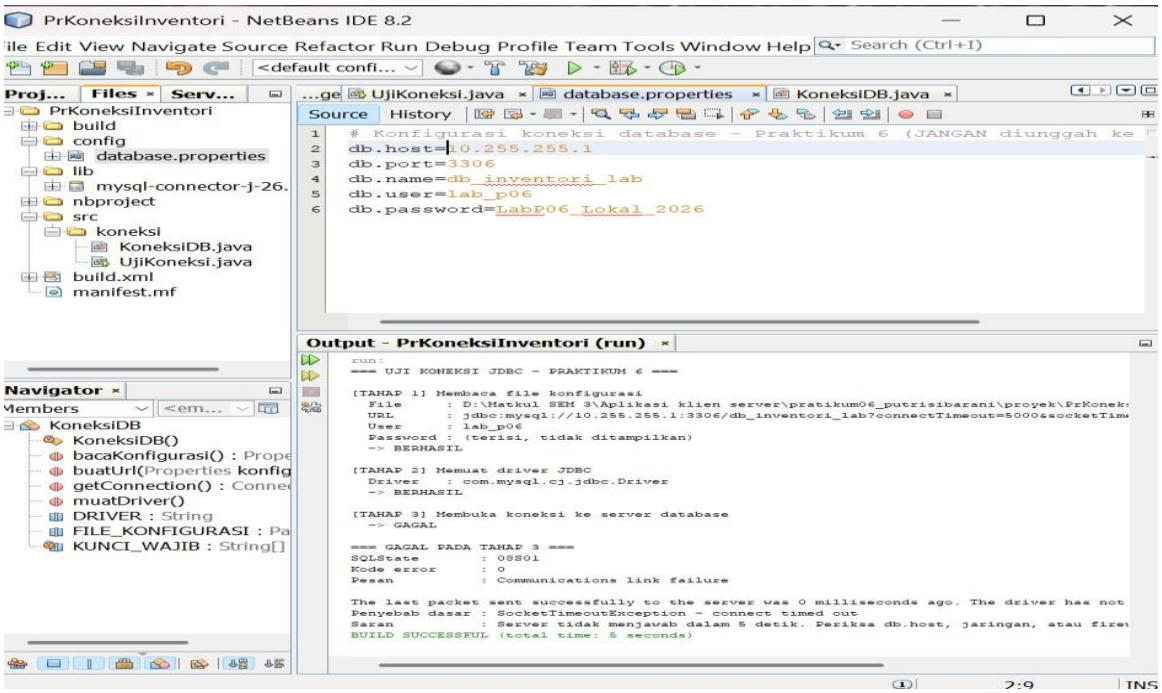
Ubah	Di database.properties: db.port=3306 menjadi db.port=3307
Hasil yang diharapkan	**Sama dengan E1**: Tahap 3, 08S01, ConnectException - Connection refused
Kembalikan	db.port=3306 (atau port asli Anda)

4. **Pengujian yang ke-tiga (Bagian H -E 3) ->** nama host salah (Langkah 27)



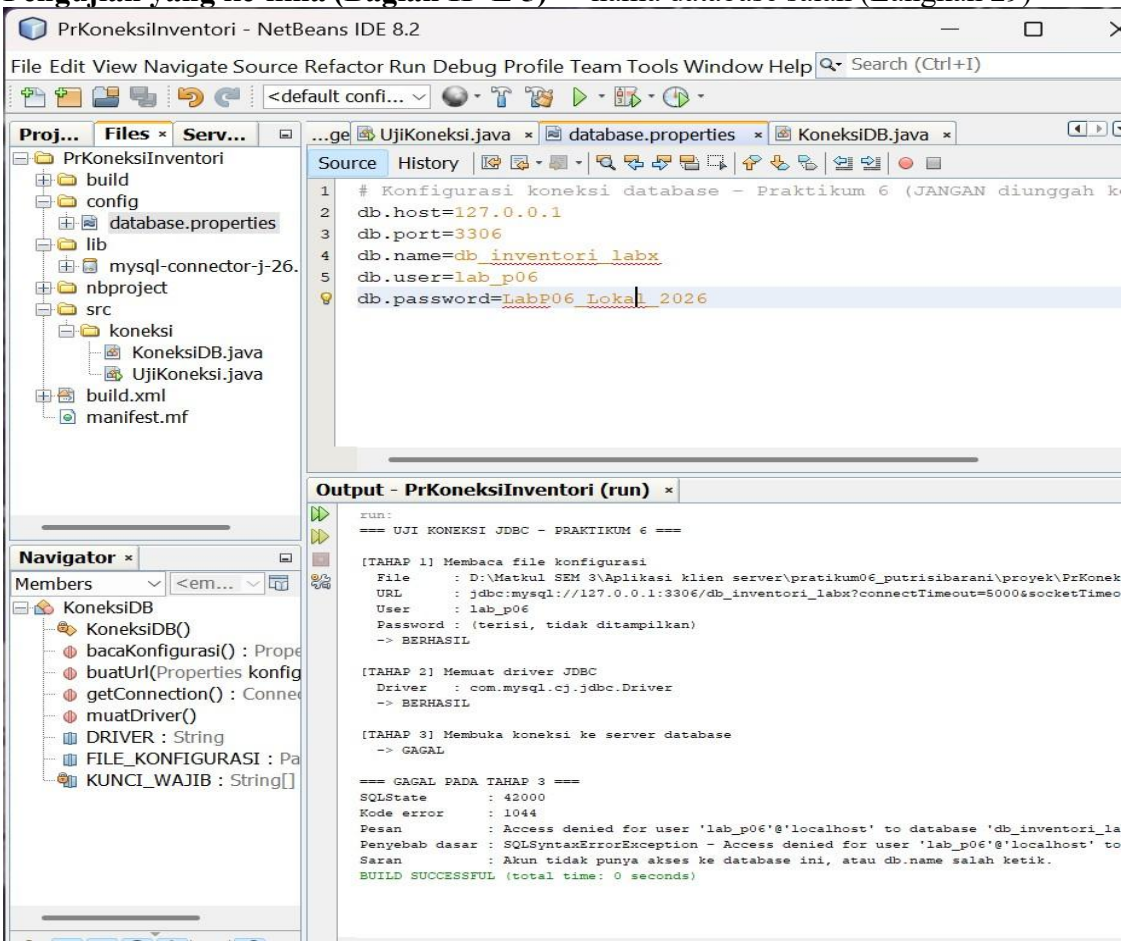
Ubah	db.host=127.0.0.1 menjadi db.host=server-lab-salah
Hasil yang diharapkan	Gagal di Tahap 3; 08S01; penyebab dasar UnknownHostException - server-lab-salah.... Kalimat setelah nama host bisa berbeda antar-Windows, misalnya No such host is known.
Kembalikan	db.host=127.0.0.1

5. **Pengujian yang ke-empat (Bagian H -E 4) ->** host tidak menjawab (Langkah 28)



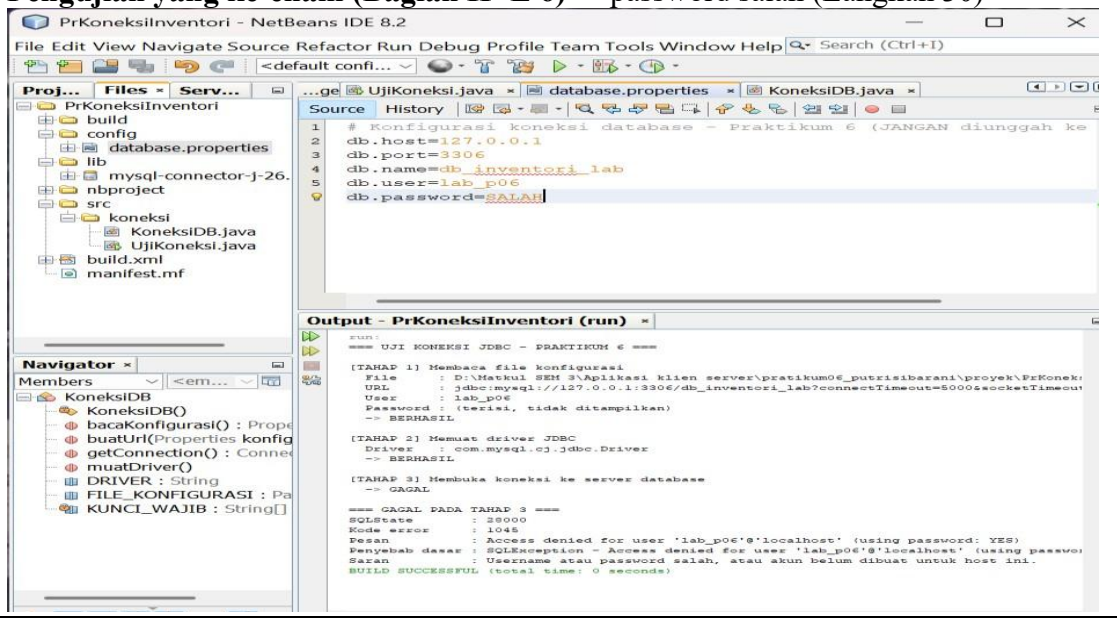
Ubah	db.host=127.0.0.1 menjadi db.host=10.255.255.1
Hasil yang diharapkan	Program menunggu sekitar **5 detik** (sesuai connectTimeout=5000), lalu gagal di Tahap 3 dengan SocketTimeoutException - connect timed out. Pada sebagian jaringan, hasilnya bisa langsung ConnectException atau NoRouteToHostException. Catat apa adanya.
Kembalikan	db.host=127.0.0.1

6. **Pengujian yang ke-lima (Bagian H -E 5) ->** nama database salah (Langkah 29)



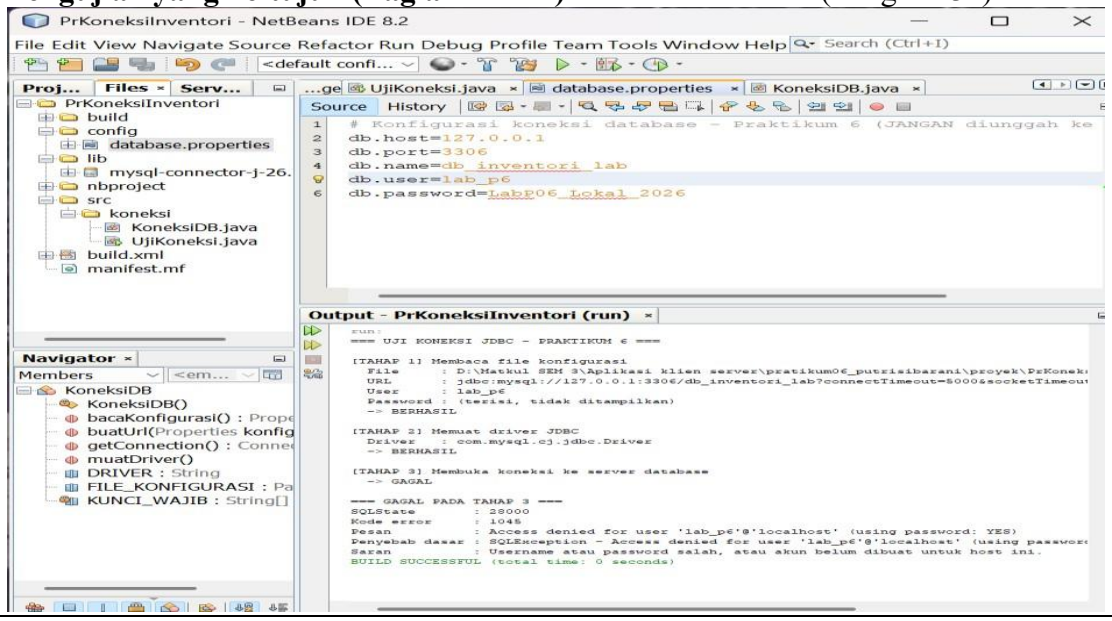
Ubah	db.name=db_inventori_lab menjadi db.name=db_inventori_labx
Hasil yang diharapkan	Gagal di Tahap 3; SQLState 42000; kode 1044; pesan Access denied for user 'lab_p06'@'localhost' to database 'db_inventori_labx'
Kembalikan	db.name=db_inventori_lab

7. **Pengujian yang ke-enam (Bagian H -E 6) ->** password salah (Langkah 30)



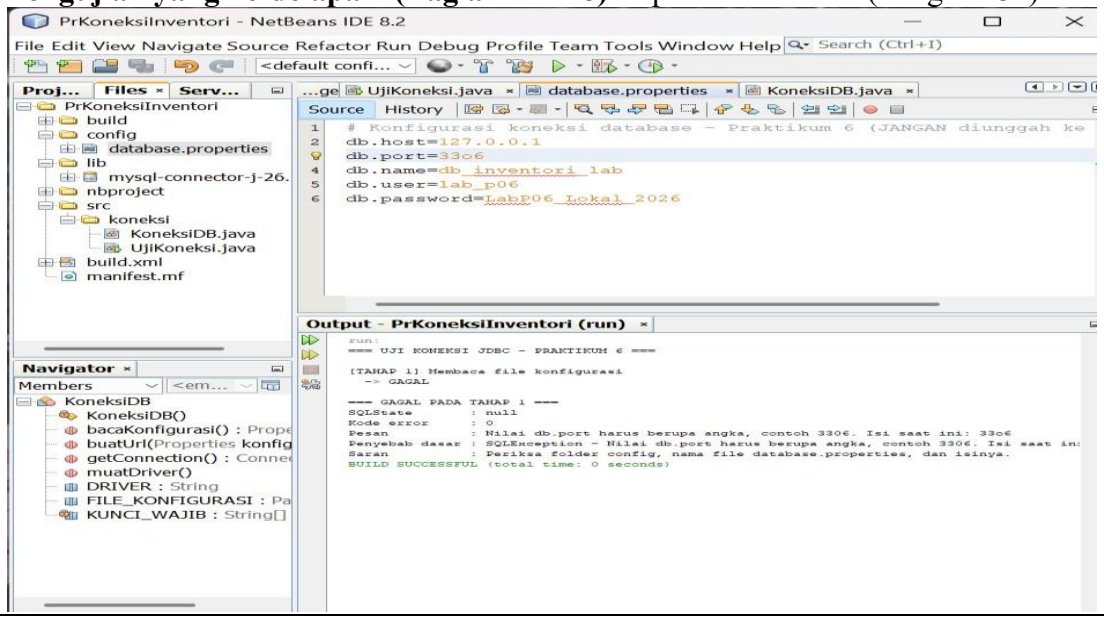
Ubah	Nilai db.password menjadi SALAH
Hasil yang diharapkan	Gagal di Tahap 3; SQLState 28000; kode 1045; pesan Access denied for user 'lab_p06'@'localhost' (using password: YES)
Kembalikan	Password yang benar

8. **Pengujian yang ke-tujuh (Bagian H -E 7) =>** username salah (Langkah 31)



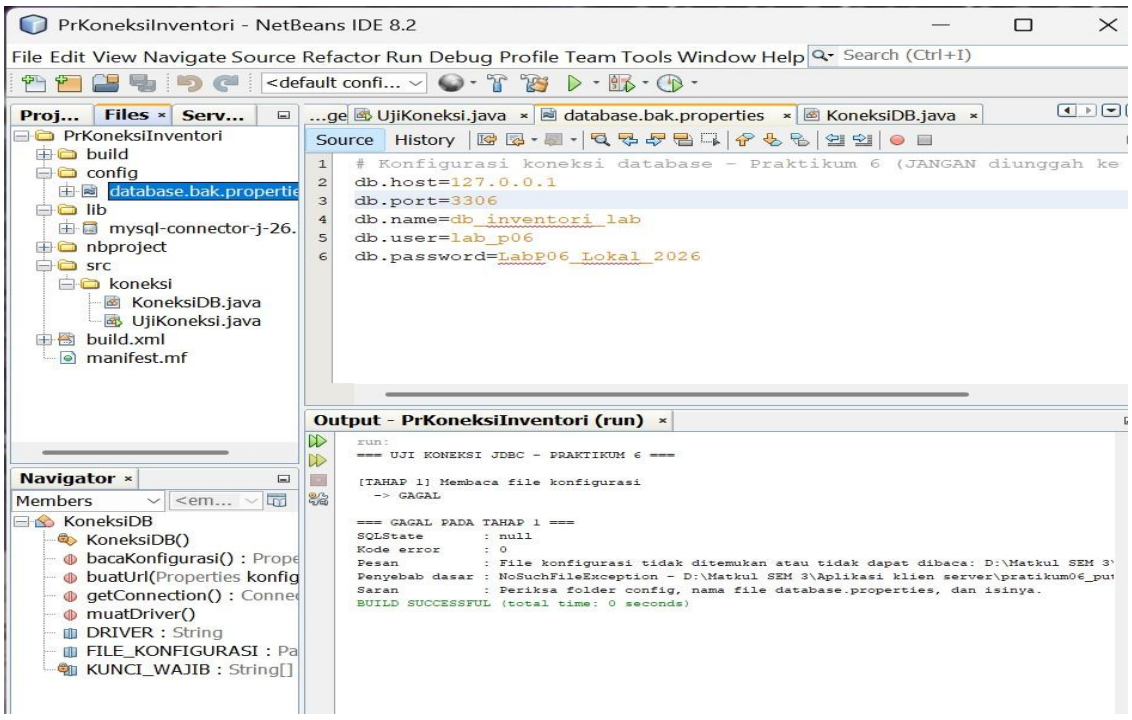
Ubah	db.user=lab_p06 menjadi db.user=lab_p6
Hasil yang diharapkan	**Sama dengan E6**: 28000, 1045
Kembalikan	db.user=lab_p06

9. **Pengujian yang ke-delapan (Bagian H -E 8) ->** port berisi huruf (Langkah 32)

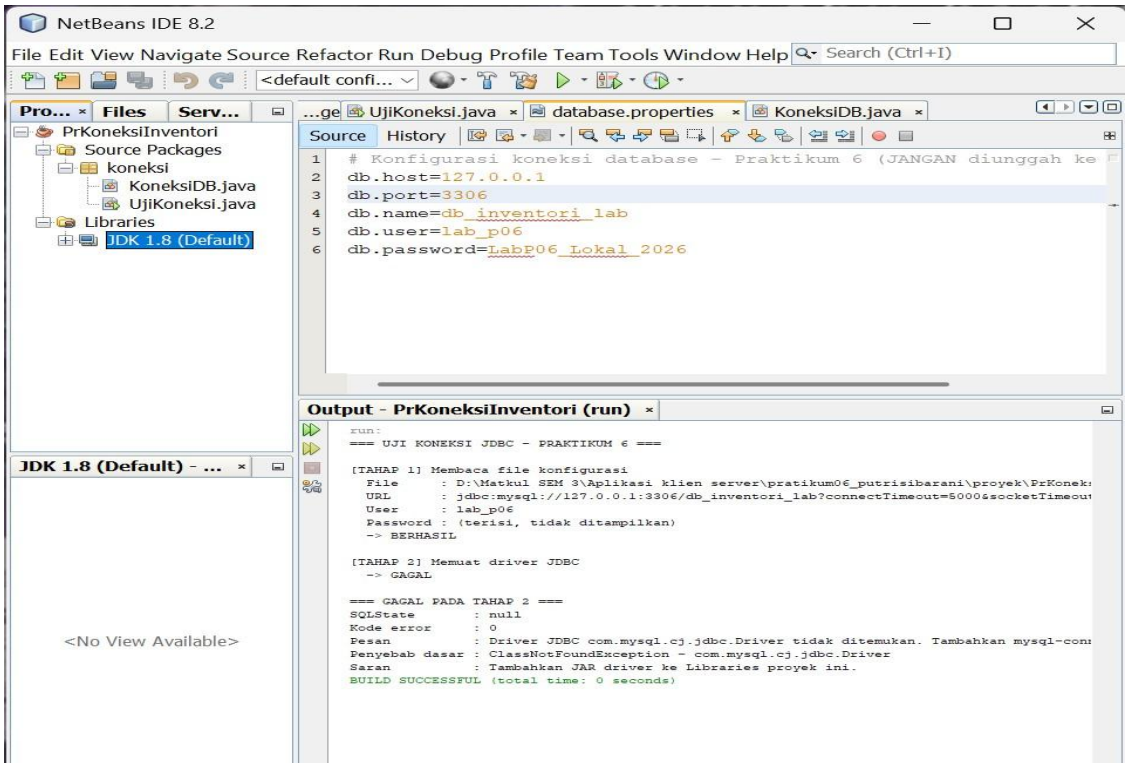


Ubah	db.port=3306 menjadi db.port=33o6 (huruf **o**, bukan angka nol)
Hasil yang diharapkan	Gagal di **Tahap 1**; SQLState null; pesan Nilai db.port harus berupa angka...
Kembalikan	db.port=3306

10. **Pengujian yang ke-sembilan (Bagian H -E 9) ->** file konfigurasi hilang (Langkah 33)

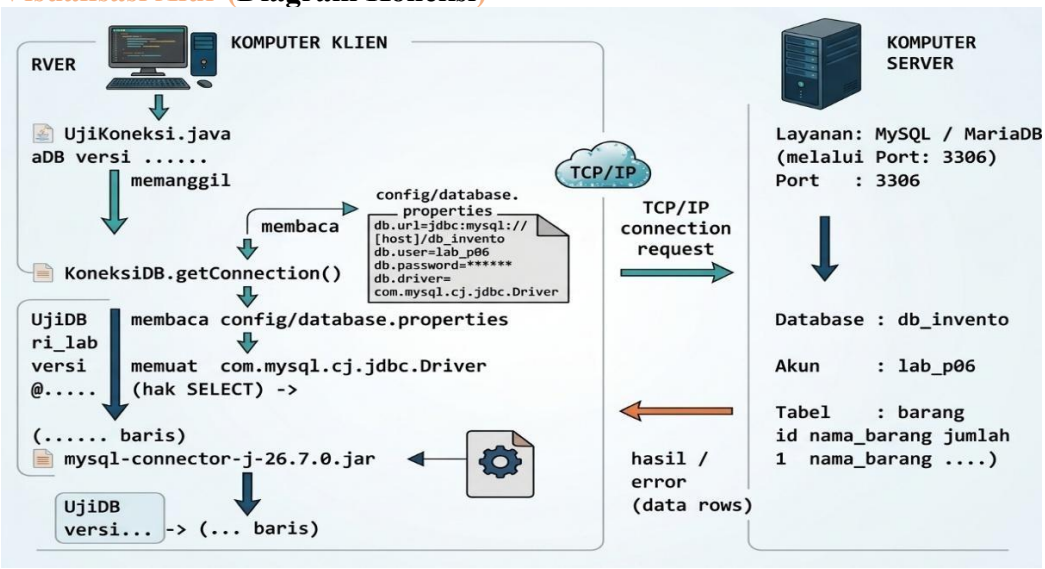


11. **Pengujian yang ke-sepuluh (Bagian H -E 10) ->** river dilepas dari Libraries (Langkah 34)



Ubah	Tab **Projects** → buka **Libraries** → klik kanan mysql-connector-j-26.7.0.jar → **Remove**
Hasil yang diharapkan	Gagal di **Tahap 2**; pesan Driver JDBC com.mysql.cj.jdbc.Driver tidak ditemukan...; penyebab dasar ClassNotFoundException
Kembalikan	Ulangi **Langkah 16** (Add JAR/Folder). File JAR di folder lib tidak terhapus; yang dihapus hanya referensinya.

Visualisasi Alur (Diagram Koneksi)



1. *Apa perbedaan peran java.sql dan file mysql-connector-j-26.7.0.jar?(Langkah persiapan)*

Jawaban :

- ✓ **java.sql**: Merupakan pustaka/paket standar (*built-in API*) yang disediakan langsung oleh Java Standard Edition (SE). Isinya berupa antarmuka (interface) dan kelas umum—seperti Connection, Statement, dan ResultSet—yang menentukan *standar cara berinteraksi* dengan basis data secara umum.
- ✓ **mysql-connector-j-26.7.0.jar**: Merupakan pustaka pihak ketiga (*external driver library*) spesifik yang dibuat oleh MySQL. File ini berisi implementasi konkret dari antarmuka java.sql agar aplikasi Java benar-benar dapat berkomunikasi secara langsung menggunakan protokol khas MySQL/MariaDB.

2. *Sebutkan bagian-bagian URL JDBC yang Anda pakai beserta artinya.? (terletak dilangkah konfigurasi koneksi pertama)*

URL: jdbc:mysql://127.0.0.1:3306/db_inventori_lab?connectTimeout=5000&socketTimeout=10000

Jawaban :

- ✓ **dbc:mysql://**: Menunjukkan protokol bahwa koneksi menggunakan JDBC dengan *driver* MySQL.
- ✓ **127.0.0.1**: Alamat IP host tempat server database berada (localhost / komputer lokal).
- ✓ **3306**: Nomor port yang digunakan oleh layanan server MySQL.
- ✓ **db_inventori_lab**: Nama database spesifik yang akan diakses.
- ✓ **connectTimeout=5000**: Batas waktu maksimal (dalam milidetik / 5 detik) untuk mencoba membuat koneksi awal ke database sebelum *timeout*.
- ✓ **socketTimeout=10000**: Batas waktu maksimal (10 detik) menunggu respons data dari server melalui koneksi yang sudah terbentuk.

3. *method apa saja yang dipanggil getConnection() sebelum DriverManager.getConnection(...), dan bagaimana urutannya?*

Jawaban :

Berdasarkan mekanisme standar pemrograman Java Database Connectivity (**JDBC**), sebelum method DriverManager.getConnection(...) berhasil mengembalikan sebuah koneksi database, kelas DriverManager secara internal akan memanggil beberapa method untuk memeriksa dan mencocokkan driver yang sesuai.

Berikut adalah method-method yang dipanggil beserta urutannya:

I. **DriverManager.getDrivers()** (atau iterasi internal pada daftar driver yang terdaftar)

   a. **Penjelasan:** DriverManager akan mengambil atau memeriksa daftar semua driver

JDBC yang telah terdaftar (diregistrasikan) di dalam sistem.

II. **Driver.acceptsURL(url)**

a. **Penjelasan:** DriverManager akan melakukan perulangan (*looping*) terhadap setiap driver yang ada di dalam daftar tersebut. Untuk setiap driver, method acceptsURL(String url) milik interface Driver akan dipanggil untuk mengecek apakah driver tersebut mengenali dan bisa menangani format URL database yang Anda masukkan (misalnya: jdbc:mysql://... atau jdbc:postgresql://...).

III. **Driver.connect(url, info)**

a. **Penjelasan:** Setelah menemukan driver yang mengembalikan nilai true pada method acceptsURL, DriverManager akan langsung memanggil method connect(...) dari driver spesifik tersebut untuk membuat koneksi riil ke database. Di dalam method connect() inilah DriverManager.getConnection(...) akhirnya mendapatkan objek Connection yang Anda butuhkan.

4. *E1 dan E2 menghasilkan pesan yang sama. Bagaimana cara memastikan penyebab sebenarnya?(Langkah Simulasi Error & Troubleshooting Koneksi (Skenario Error Handling)*

Jawaban :

Pesan error seperti *Communications link failure* bersifat umum ketika aplikasi gagal terhubung ke database. Untuk memastikan penyebab pastinya:

1) Periksa panel kontrol XAMPP untuk memastikan apakah layanan **MySQL sedang aktif (Start)** atau mati total (penyebab E1).

2) Periksa apakah ada perubahan konfigurasi port (misalnya database beralih ke port 3307) atau kesalahan penulisan port pada string koneksi Anda (penyebab E2).

3) Periksa file log error MySQL untuk melihat detail kendala di sisi server.

5. *Mengapa server tidak membedakan "username salah" dari "password salah"?(Langkah Keamanan Autentikasi / Manajemen Akses Basis Data)*

Jawaban :

Demi alasan keamanan (security). Jika server memberikan pesan yang berbeda (misalnya: "Username tidak ditemukan" vs "Password salah"), peretas atau pihak yang tidak berwenang dapat memanfaatkan informasi tersebut untuk menebak-nebak akun valid yang terdaftar di dalam sistem secara bertahap (username enumeration attack). Dengan memberikan pesan generik yang sama, akun database tetap lebih terlindungi.

6. *Apa bukti bahwa try-with-resources sudah menutup koneksi? Apa akibatnya jika koneksi tidak pernah ditutup?(Langkah Implementasi Statement & Eksekusi Query SQL)*

Jawaban :

I. Bukti: Program dapat berjalan lancar dan mengeksekusi blok kode hingga selesai, di mana secara otomatis struktur di dalam blok try(...) yang mengimplementasikan AutoCloseable (seperti objek Connection, Statement, dll) akan langsung memanggil metode .close() begitu blok kode selesai dijalankan.

II. Akibat jika tidak ditutup: Sumber daya database (database resources) akan terus terbuka dan menumpuk, menyebabkan jumlah kuota koneksi aktif di server semakin berkurang (connection leak). Jika dibiarkan, hal ini dapat membuat aplikasi mengalami out of

connections dan mengganggu kinerja server database secara keseluruhan.

7. *Mengapa password disimpan di database.properties dan tidak ditulis di kode Java?(Langkah Pemisahan Konfigurasi / Best Practice Pengembangan Aplikasi*

Jawaban :

Untuk menerapkan prinsip pemisahan konfigurasi (separation of concerns) dan keamanan kode. Jika password ditulis secara langsung (hardcoded) di dalam file kode sumber Java, maka kredensial tersebut akan ikut terekspos jika kode program diunggah ke repositori publik (seperti GitHub) atau dibagikan ke orang lain. Dengan menggunakan file database.properties, data sensitif dapat dipisahkan dan dikecualikan dari version control.